

PIMDup: An Optimized Deduplication Design on a Real Processing-in-Memory System

Chun-Le Yeh
Department of CSIE
National Cheng Kung University
Tainan, Taiwan (R.O.C.)
p76101356@gs.ncku.edu.tw

Liang-Chi Chen
Department of CSIE
National Taiwan University
Taipei, Taiwan (R.O.C.)
d12922012@csie.ntu.edu.tw

Chien-Chung Ho
Department of CSIE
National Cheng Kung University
Tainan, Taiwan (R.O.C.)
ccho@gs.ncku.edu.tw

Yu-Ming Chang
Wolley Inc.
Hsinchu, Taiwan (R.O.C.)
san@wolleytech.com

Da-Wei Chang
Department of CSIE
National Cheng Kung University
Tainan, Taiwan (R.O.C.)
davidchang@csie.ncku.edu.tw

Abstract—Data deduplication enhances storage efficiency through non-destructive compression but is often hindered by the chunking process, which requires scanning the entire dataset. While traditional methods leveraging conventional architectures and hardware accelerators (e.g., GPUs and FPGAs) have been developed to address this issue, they continue to face challenges related to excessive data movement and associated performance degradation. These limitations stem from the von Neumann architecture, where computation and storage are separated in a processor-centric design, necessitating multiple memory hierarchy traversals and causing inefficiencies. To overcome these challenges, we explore UPMEM’s DPU, a processing-in-memory (PIM) technology that reduces data movement by performing computations directly within memory. However, designing a deduplication system for DPUs presents unique obstacles, including restricted inter-DPU data sharing, the absence of native multiplication support, and significant DPU-CPU communication overhead. In response, we propose PIMDup, a DPU-optimized deduplication system that addresses these constraints through efficient parallelization, DPU-friendly chunking techniques, and reduced data transfer volumes. Experimental results demonstrate that PIMDup improves chunking performance without compromising deduplication accuracy, achieving a 1.67× speedup over CPU-based systems while maintaining 100% result consistency.

Index Terms—Deduplication system, processing-in-memory, UPMEM DPU

I. INTRODUCTION

In recent years, we have entered the era of the information explosion. This growing disparity places significant strain on storage systems, which must manage increasingly larger volumes within constrained capacities. To address these challenges, various solutions have been proposed, including distributed storage systems [1], [2], parallel architectures [3], [4], and hybrid systems leveraging solid-state drives (SSDs) [5], [6], which offer performance advantages over traditional hard disks. However, despite these advances, the fundamental issue persists: the sheer volume of data must be reduced. As a result, techniques for lowering storage costs are becoming increasingly critical.

Compression [7] and data deduplication [8] are two primary methods for enhancing storage efficiency. Compression reduces the size of individual files by identifying and re-encoding redundancy, with the degree of reduction depending

on whether the process is lossy or lossless. In contrast, data deduplication identifies and eliminates duplicate data segments by referencing existing copies, conserving write bandwidth, and improving storage utilization. In data centers, deduplication typically achieves higher compression ratios than traditional compression while preserving non-destructive characteristics, allowing full restoration of original data. The deduplication process consists of four key steps [9]: (1) files are divided into chunks, the smallest units for deduplication; (2) each chunk is assigned a unique hash value (fingerprint) using a secure hash algorithm; (3) fingerprints are compared with existing ones to identify duplicates; and (4) duplicate chunks are replaced with references, while unique chunks and their fingerprints are stored for future reconstruction. Among these steps, *chunking—segmenting data into de-duplicatable fragments—is the most time-intensive, accounting for up to 53% of deduplication runtime* [10]. Accelerating chunking through processing-in-memory (PIM) technologies offers a promising approach to improving overall deduplication performance.

Processing-in-Memory (PIM) technology addresses data-intensive challenges by enabling computation directly within memory, reducing data movement overhead [11]. The UPMEM DPU, the first commercially available PIM solution [12], has demonstrated significant potential for tackling memory-bound workloads [13]–[19]. By integrating hundreds of processors within DRAM DIMMs, UPMEM DPUs allow parallelized, on-site data processing, making them well-suited for accelerating deduplication workflows. However, two major limitations constrain UPMEM DPU’s effectiveness. First, data transfer remains dependent on the host CPU, as DPUs cannot directly access storage, limiting the offloading of the entire deduplication process. Second, UPMEM DPUs lack inter-processor communication [20], [21], complicating the generation of consistent chunking results across multiple processors. Despite these challenges, UPMEM DPUs offer substantial advantages, provided that data transfer costs are carefully managed and deduplication accuracy is maintained. Motivated by these opportunities and limitations, we propose PIMDup, a deduplication system explicitly optimized for DPU archi-

textures. PIMDup addresses the unique challenges of DPU-based deduplication through a carefully designed system that balances hardware constraints with deduplication principles. This work aims to: (1) identify and address the technical challenges of implementing chunking on DPUs, (2) design a deduplication framework tailored for DPU hardware, and (3) evaluate the system's performance and deduplication accuracy through comprehensive experiments.

II. BACKGROUND

A. Data Deduplication

Deduplication operates across files by identifying and eliminating redundant data segments, ensuring only unique data is stored. By referencing duplicate data instead of storing it repeatedly, deduplication significantly reduces storage volume, delivering high compression ratios while retaining lossless, non-destructive characteristics. Due to these advantages, deduplication has become a widely adopted technique in modern storage systems, offering both bandwidth savings and improved storage efficiency.

The deduplication process typically consists of four main steps: chunking, fingerprinting, indexing, and storage, as illustrated in Figure 1. (1) **Chunking**: Each file is divided into chunks, either of fixed or variable size, depending on the chosen chunking strategy. (2) **Fingerprinting**: A secure hash algorithm, such as SHA-1, MD5, or SHA-256 [22]–[24], is used to compute a unique fingerprint (hash value) for each chunk, serving as its identifier. (3) **Indexing**: The generated fingerprints are compared against an index to check for duplicates. If a matching fingerprint is found, the chunk is identified as redundant; otherwise, it is considered unique. (4) **Storage**: Unique chunks are stored, and their fingerprints are added to the index. For duplicate chunks, the system creates references to the already stored chunks, avoiding redundant storage. When reading data in a deduplication system, the original file is reconstructed by retrieving both unique chunks and references to duplicates. Using metadata to identify the correct chunk sequence, the system reassembles the data in its original form. This ensures that deduplicated data can be seamlessly restored when needed, maintaining usability and reliability.

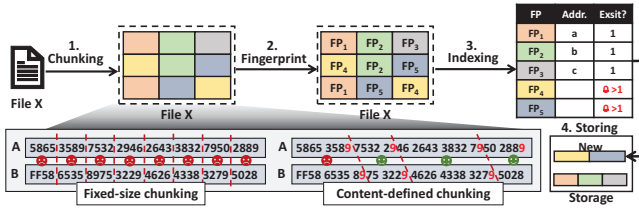


Fig. 1. The work-flow of deduplication process.

Among the four steps, chunking is particularly critical, as it determines how data is segmented into the smallest units for deduplication. An effective chunking strategy maximizes storage efficiency by accurately identifying redundant data. Conversely, a poorly designed chunking method may fail to detect similarities, resulting in suboptimal deduplication ratios and wasted computational effort. As shown in Figure 1,

chunking methods are broadly classified into two categories: fixed-size chunking [25] and content-defined chunking [26]. Fixed-size chunking method has a well-known limitation, the boundary-shift problem [27], [28], which results in reduced deduplication ratios. To address this, content-defined chunking (CDC) has been developed. CDC uses specific patterns within the file's content as chunk boundary markers. Combining these markers with predefined maximum and minimum chunk sizes helps determine chunk boundaries, as seen in the example of number 9 in Figure 1. The data between two chunk boundaries forms a chunk. Since the content itself determines these boundaries, CDC is capable of detecting redundant data even in files with slight variations, such as insertions or deletions. Therefore, this work adopts content-defined chunking to optimize the chunking process in deduplication systems.

B. UPMEM PIM DIMM

The UPMEM DRAM DIMM is a widely adopted commercially available Processing-in-Memory (PIM) product [12]. It integrates multiple in-DRAM processors, known as DRAM Processing Units (DPUs), directly into the memory. Unlike the traditional von Neumann architecture, which separates processing unit (CPU), memory, and storage, the UPMEM PIM DIMM enables computational capabilities at the memory tier, effectively breaking the conventional von Neumann bottleneck, i.e., data movement overheads. As shown in Figure 2, each UPMEM PIM DIMM comprises 16 PIM DRAM chips, and they are organized into two ranks of 8 chips each. Every chip contains 8 DPUs, resulting in 64 DPUs per rank. Each DPU operates as an independent processor and is equipped with a 64 MB DRAM bank, referred to as MRAM. In addition, each DPU has 64 KB of working memory (WRAM), functioning as a scratchpad, and 24 KB of instruction memory (IRAM) to store encoded instructions.

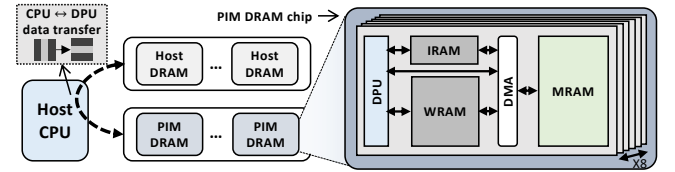


Fig. 2. The architecture of UPMEM DPU.

While DPUs enable memory-level computation and provide high external communication bandwidth via DDR slots, their architecture poses several challenges [20]. First, each DPU can only access its own MRAM, with no capability for direct data sharing or communication between DPUs, even within the same rank. Any inter-DPU communication must be mediated by the host CPU, introducing significant latency. Second, the limited capacity of WRAM necessitates careful memory management to minimize data transfers between WRAM and MRAM, as frequent transfers can negatively impact performance [15]. Third, DPUs cannot directly access data stored on disk; instead, data must first be transferred to the host CPU, which then forwards it to the DPU's MRAM. This process introduces additional overhead, such as matrix transpositions, which can further reduce system efficiency. Finally, the computational performance of DPUs is generally

lower than that of the host CPU, particularly for operations such as multiplication and division. These computational limitations can lead to increased execution latency during complex workloads, as further discussed in Section II-C.

C. Observation and Motivation

As outlined in earlier sections, DPUs cannot directly access data from storage nor share data among themselves. However, the deduplication process, particularly the indexing stage, relies on a global table for storing all fingerprints and involves storage I/O operations. Additionally, chunking—responsible for nearly half of the deduplication runtime—is a highly data-intensive task. Consequently, chunking presents a more suitable target for DPU acceleration. Based on this observation, large files can be divided into multiple segments and allocated to UPMEM PIM DIMMs for chunking within the DPUs, while the CPU handles the remaining tasks. Such a management procedure is illustrated in Figure 3: (1) Each file segment undergoes chunking on the DPUs. (2) The resulting chunking data is sent back to the host CPU. (3) The host performs fingerprinting, indexing, and storage I/O while the DPUs concurrently process chunking tasks for subsequent file segments (looping back to 1). This pipelined approach overlaps DPU and CPU operations, maximizing resource utilization.

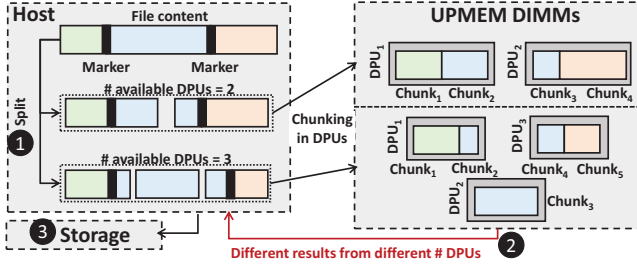


Fig. 3. A naïve design of a DPU-based deduplication.

This straightforward design encounters several critical challenges. First, the chunking process relies on a sliding window mechanism that scans each file segment, computing a hash at each position. When the hash matches a predefined value, a marker is placed to denote a potential chunk boundary. However, if a marker spans two DPUs, it cannot be accurately identified. Once markers are determined, each DPU uses them to define chunk boundaries, adhering to constraints on maximum and minimum chunk sizes. A key issue arises at the boundaries of file segments, where the DPU may fail to identify markers accurately, treating the remaining portion as a single chunk. This inconsistency leads to varying chunking results depending on the number of DPUs utilized. For instance, as illustrated in Figure 3, processing a file with 2 DPUs produces 4 chunks, whereas using 3 DPUs results in 5 chunks, undermining the stability and effectiveness of the deduplication process. Second, existing studies highlight a disparity in data transfer bandwidth between the CPU and DPUs. Specifically, the bandwidth for sending data from the host CPU to the DPUs exceeds that for returning processed results from DPUs to the CPU [20]. In the naïve design depicted in Figure 3, this imbalance results in smaller data volumes being transmitted over the faster channel (CPU to

DPU), while larger processed data sets are sent over the slower channel (DPU to CPU). This inefficiency increases data transfer times and ultimately degrades overall system performance.

Moreover, the chunking process relies on hash functions to identify markers within file content. However, state-of-the-art hashing algorithms are not well-suited for execution on DPUs due to their reliance on polynomial equations that involve a significant number of multiplication operations. DPUs exhibit a substantial performance gap between addition and multiplication operations due to hardware limitations. As illustrated in Figure 4, the throughput for addition/subtraction operations on 32-bit integers reaches approximately 58.33 MOPS, whereas the throughput for multiplication operations is considerably lower. This disparity becomes even more pronounced for 64-bit integers, where addition/subtraction operations significantly outperform multiplication. These performance trends are consistent with benchmarks from prior research, which demonstrate that DPUs are poorly suited for tasks requiring extensive multiplication.

To address these challenges and the inherent hardware limitations of DPUs, we have formulated several key design objectives. First, all potential marker positions within a file must be identified accurately, even when markers span across multiple DPUs, to ensure precise chunking boundaries. Second, the chunking results must remain consistent, irrespective of the number of DPUs involved, to preserve the deduplication system's capacity for effective storage reduction. Third, considering the inefficiency of DPUs in performing multiplication-heavy operations, it is crucial to develop a chunking method tailored to the computational strengths of DPUs, thereby enhancing execution performance. Finally, the disparity in data transfer speeds between the DPUs' input and output channels must be addressed through optimized data transfer mechanisms to minimize performance bottlenecks. With these objectives in mind, this work introduces *PIMDup*, a DPU-based deduplication system specifically designed to capitalize on the in-memory processing capabilities and parallelism of DPUs. *PIMDup* is structured to address the outlined challenges while ensuring consistency and efficiency across a range of DPU configurations.

III. PROPOSED PIMDUP DESIGN

A. Overview

Content-defined chunking is essential for efficient data deduplication but is also the most time-consuming step. To address this, we employ DPUs as parallel workers to accelerate the process. To ensure consistent chunking results across varying DPU counts, we propose an *anti-splitting file segmentation* (ASFS) method to identify all markers in the input file accurately. A *chunk boundary vector* mechanism records marker positions in DPUs, which are merged on the host CPU to define global chunk boundaries based on conditions like maximum chunk size (see Section III-B). Additionally, an *encoded boundary vector* compresses boundary data, reducing data transfer to one-eighth of the input file size (see Section III-C). To enhance performance, a *DPU-friendly chunking method* avoids inefficient computational patterns, improving speed without sacrificing accuracy (see Section III-D).

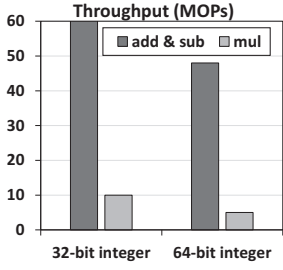


Fig. 4. A huge gap between arithmetic performances in the DPU.

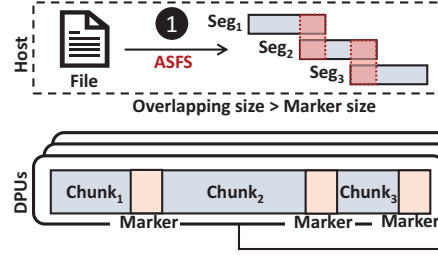


Fig. 5. Overview of proposed PIMDup.

Figure 5 outlines the workflow of the proposed *PIMDup* system. The host CPU first segments the input file using ASFS ① and distributes segments to the DPUs ②. DPUs perform chunking using the DPU-friendly method, identifying markers and storing them in a chunk boundary vector ③. These vectors are encoded into 8-bit integers and sent to the host CPU ④, which reconstructs the vectors, aligns them with the input file, and applies constraints to finalize chunk boundaries ⑤. The host CPU then completes fingerprinting and indexing ⑥. While the CPU processes chunking results, the DPUs can immediately begin processing the next file segment, ensuring efficient parallelism and maximizing throughput. This pipeline reduces data transfer overhead and accelerates the deduplication process.

B. Anti-Splitting File Segmentation (ASFS)

DPUs lack a direct mechanism for sharing data among themselves. Consequently, if a file is split into mutually exclusive segments and distributed across multiple DPUs, a marker could fall on the boundary between segments, resulting in an incorrect marking of chunk boundary candidates. This error compromises the deduplication system’s efficiency, reducing its bandwidth and space-saving benefits. To address this issue, we propose the anti-splitting file segmentation (ASFS) technique, which introduces overlapping regions between file segments during segmentation. The length of this overlap is set to exceed the size of the designated markers, as illustrated in Figure 5. In cases where a marker lies on the boundary between two DPUs, one of the DPUs will contain a complete copy of the marker within its overlapping segment. This ensures that the marker can be correctly identified during content-defined chunking, preventing errors in chunk boundaries. By using ASFS, we can guarantee that all markers are accurately detected, regardless of the number of active DPUs. In large-scale systems with thousands of DPUs, particularly in scenarios where the UPMEM PIM DIMM’s yield varies, the number of DPUs assigned to a task may change dynamically. ASFS addresses this challenge by ensuring consistent chunking results across different DPU allocations, even when a varying number of DPUs processes the same file.

It is worth mentioning that the overlapping segment caused by applying ASFS does not incur significant memory duplicated overhead. Given that typical marker sizes are around 32 bytes, the extra memory required for overlapping segments is negligible. As these markers are often numerous

and distributed randomly throughout the file, ASFS significantly enhances performance. By ensuring that each marker is contained fully within at least one segment, ASFS prevents costly chunking errors and preserves the deduplication system’s bandwidth and storage efficiency. The benefits of this solution scale effectively, particularly in systems with high marker density, leading to improved overall performance with minimal trade-offs in space utilization.

C. Encoded Boundary Vector

The primary cause of inconsistency in chunking results is the inability of DPUs to communicate, which prevents them from having a “global view” when determining chunk boundaries. This limitation can result in some data segments being arbitrarily defined as chunks, even if they lack proper boundaries on both ends. To address this issue, we propose using a chunk boundary vector. As shown in Figure 5, after each DPU completes its chunking process, it generates a chunk boundary vector that records the positions of special markers, which are potential candidates for chunk boundaries. At this stage, these segments are not yet defined as chunks. Instead, each DPU transmits its chunk boundary vector to the host CPU. The host CPU then aggregates these individual vectors to create a “global vector” that corresponds to the entire file, capturing all potential chunk boundaries. Based on this global vector and predefined parameters, such as maximum and minimum chunk sizes, the host CPU determines the final chunk boundaries. Compared to the naïve design in Figure 3, where each DPU independently defines chunk boundaries, this approach allows the host CPU to make boundary decisions with a comprehensive, file-wide perspective, thereby ensuring consistency in the chunking results. Thus, it can eliminate the problem shown in Figure 3, that is, the inconsistent chunking outcomes for the same file when it is distributed across varying numbers of DPUs.

As discussed in Section II-C, the data transfer channels between the CPU and DPUs have differing speeds, with the output channel (DPU-to-CPU) being slower than the input channel (CPU-to-DPU). This difference makes it crucial to minimize the amount of data sent through the slower output channel. As shown in ③ of Figure 5, to optimize data transfer efficiency, we encode the chunk boundary vector as a binary array, where each ‘1’ represents the presence of a marker. By grouping every 8 bits of this array into a single binary integer, we can represent the chunk boundary information

much more compactly. Each element in the chunk boundary vector corresponds to the outcome of the sliding window hash operation, indicating whether the calculated hash value at each byte matches the designated marker. This setup enables the chunking results for every 8 bytes of the file to be encoded as a single 8-bit integer, which is then transferred from the DPUs back to the main memory. Compared to a non-encoded chunk boundary vector, where each byte in the vector represents whether a corresponding byte in the input file contains a marker, this binary encoding is much more efficient. By encoding each marker presence as a single bit, we reduce the data size that needs to be transferred. This approach ultimately reduces the transfer data size to one-eighth of the original, as it efficiently captures marker information within each byte.

D. DPU-Friendly Chunking Method

As illustrated in Figure 4, DPU performance is critical for determining processing speed. However, conventional chunking algorithms often rely on hashing with polynomial functions, which significantly hinder DPU execution efficiency. A straightforward solution might involve using a pre-defined table within each DPU to store all possible string values. However, this approach is impractical due to the limited 64 MB of MRAM available per DPU, which must simultaneously accommodate data storage and results for transfer back to the host, leaving insufficient memory for such a table.

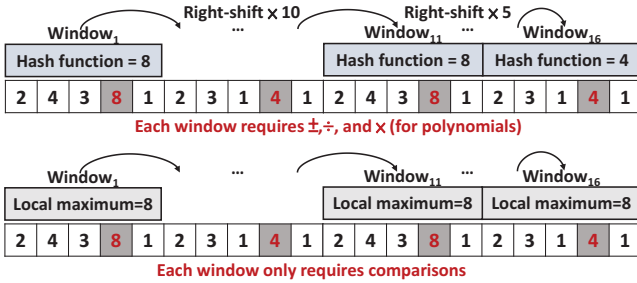


Fig. 6. Chunking by Rabin-based hashing v.s DPU-friendly chunking method

To overcome this limitation, we propose a non-hashing, DPU-friendly chunking method based on local-maximum-byte chunking [29]. This method employs a sliding window of predefined size to scan the data. As the window moves, the middle byte is checked to determine if it is the maximum within the window. If it is not, the window advances. If it is, the boundary between the middle byte and the preceding one is marked as a candidate chunk boundary. These boundary positions are then recorded using the encoded boundary vector method, ensuring efficient transfer back to the host. To align with the granularity of the encoded boundary vector, the proposed method uses a 32-byte sliding window to process the file. Unlike hashing-based approaches that rely on polynomial calculations and require multiplication and division operations, the local-maximum-byte chunking method only involves comparing byte values within the window. This comparison-based approach eliminates the computational overhead associated with polynomial functions. As shown in Figure 6, Rabin-based hashing methods involve numerous multiplications for

polynomial functions, whereas the DPU-friendly chunking method relies solely on comparisons. These comparisons, implemented using subtraction, are highly efficient for DPUs due to their hardware design, further enhancing overall processing performance.

IV. EXPERIMENT

A. Experimental Setup

In this section, we evaluate the performance of the proposed PIMDup design from multiple perspectives. The analysis of the experimental results is conducted based on the following metrics: (1) Internal DPU Performance: This metric assesses the benefits derived from the DPU-Friendly Chunking Method. (2) Data Transfer Bandwidth: To evaluate the effectiveness of the Encoded Boundary Vector, we measure the data transfer bandwidth between the CPU and DPUs. (3) Result Consistency: We verify that different numbers of DPUs performing chunking do not lead to inconsistent results. (4) Overall System Performance: We compare the performance of PIMDup against a traditional CPU-based deduplication system.

To demonstrate the feasibility of the PIMDup design, experiments were conducted on a real system equipped with UPMEM DPUs. Specifically, we utilized a UPMEM server with the following configuration: an Intel(R) Xeon(R) Silver 4110 CPU, 128 GB of host main memory, and 20 dual-rank PIM DIMMs, providing a total of 40 ranks and 2560 DPUs operating at @ 350 MHz. The PIMDup system was implemented in C/C++ using the UPMEM SDK¹, and DPU-side chunking logic was programmed using the LLVM-based compiler provided by the SDK. This setup enabled direct performance comparison between PIMDup and traditional CPU-based deduplication systems.

The dataset used to evaluate PIMDup comprises 30 different release versions of virtual machine image files [9], [30]. The dataset includes 6 CentOS, 5 Fedora, and 19 Debian image files, collectively amounting to approximately 30 GB. These files exhibit deduplication ratios ranging from 38% to 55%, providing a comprehensive and diverse testbed for performance evaluation. The mixture of distributions and versions allows for a robust assessment of PIMDup's performance and consistency across various data types, thereby validating its effectiveness and reliability in real-world scenarios.

B. Experimental Result

To comprehensively evaluate the scalability of the proposed PIMDup design, we incrementally increased the number of DPU ranks from 4 to 36 (with each rank consisting of 64 DPUs) and measured the total execution time required for deduplication.

Figure 7 illustrates the results, detailing the time required for chunking, fingerprinting, and indexing across varying numbers of DPU ranks. The experimental results reveal a significant reduction in the total deduplication time as the number of DPU ranks increases. Notably, chunking remains the largest contributor to the overall deduplication time, though it decreases substantially with the addition of more DPU ranks. This improvement is attributable to the parallel processing

¹<https://sdk.upmem.com>

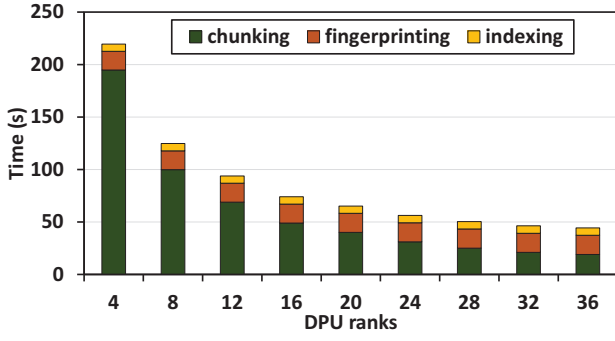


Fig. 7. The breakdown analysis for chunking, fingerprinting, and indexing time of PIMDup.

capabilities of the DPUs, which allow for more efficient handling of data chunks. In contrast, fingerprinting and indexing times show little to no improvement as the number of DPU ranks increases. This is because these tasks are executed by the CPU in PIMDup and are therefore unaffected by the scaling of DPUs. Despite this, the proposed PIMDup design ensures that the host CPU does not experience significant overhead during the integration of processed data. This demonstrates the scalability and efficiency of PIMDup in leveraging parallelism within DPUs while maintaining minimal impact on host-side operations.

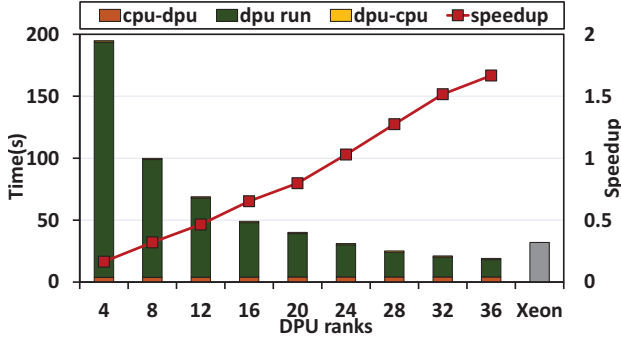


Fig. 8. The overall performance analysis and comparison for PIMDup.

Figure 8 provides several important insights into the performance of PIMDup. As the number of DPU ranks increases, the deduplication time decreases significantly, showcasing the scalability of the DPU-based implementation. Specifically, the processing time drops dramatically—from nearly 200 seconds with 4 DPU ranks to approximately 20 seconds with 36 DPU ranks. When compared to CPU-based deduplication (e.g., using Xeon processor), the advantages of our proposed PIMDup become even more apparent. At the maximum number of DPU ranks, PIMDup achieves a $1.67\times$ reduction in processing time relative to the Xeon-based system. This improvement highlights how the DPU architecture excels in managing memory-bound workloads more efficiently than traditional CPU-based systems. Moreover, the data transfer overhead between the CPU and DPUs remains minimal, thanks to the adoption of the Encoded Boundary Vector design. This mechanism allows DPUs to efficiently notify the host of chunking results using a compact representation, reducing the volume of data that

must be transferred. Consequently, the host is able to seamlessly integrate the chunking information while minimizing data transfer overhead, further enhancing the overall system efficiency.

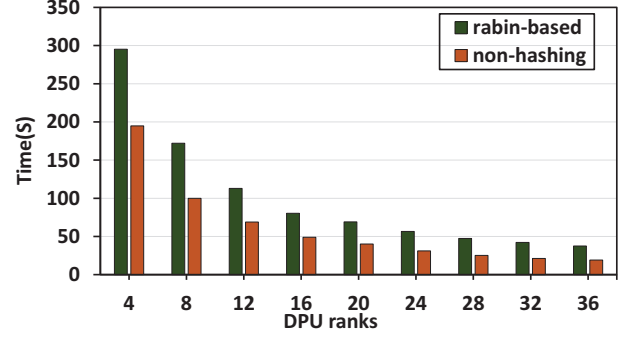


Fig. 9. The time required for rabin-DPU and non-hashing-DPU using different numbers of DPU ranks.

Figure 9 demonstrates that the DPU-friendly chunking method consistently outperforms the Rabin-based chunking method across all tested DPU configurations, regardless of the number of ranks employed. Notably, the DPU-friendly chunking method achieves an approximately 50% reduction in processing time compared to the Rabin-based approach. This substantial performance improvement stems from the inherent hardware constraints of the DPUs. As highlighted in our preliminary analysis, DPUs exhibit significantly higher throughput for addition and subtraction operations than for multiplication and division. The Rabin-based algorithm, which relies on polynomial multiplication to compute hash values, is computationally intensive and less suited to the architecture of the DPUs. In contrast, the DPU-friendly chunking method aligns better with the DPUs' operational strengths, enabling more efficient processing.

V. CONCLUSION

In this work, we present PIMDup, a deduplication system specifically optimized for DPUs, addressing critical challenges such as limited inter-DPU data sharing, the absence of native multiplication support, and the overhead associated with DPU-CPU data transfers. To overcome these issues, PIMDup introduces three key innovations: Anti-Splitting File Segmentation (ASFS) for efficient data allocation, an Encoded Boundary Vector to streamline data transfers between DPUs and the CPU, and a DPU-Friendly Chunking Method to enable high-performance execution on DPUs. Our experimental evaluation demonstrates that PIMDup achieves a $1.67\times$ speedup compared to a CPU-based deduplication system while ensuring 100% result consistency. This highlights the effectiveness and reliability of our proposed design.

VI. ACKNOWLEDGMENTS

This work was supported in part by National Science and Technology Council under grant nos. 112-2221-E-006-126-MY3 and 113-2221-E-006-217-MY3.

REFERENCES

- [1] Y. Yamato, "Use case study of hdd-ssd hybrid storage, distributed storage and hdd storage on openstack," in *Proceedings of the 19th International Database Engineering & Applications Symposium*, ser. IDEAS '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 228–229. [Online]. Available: <https://doi.org/10.1145/2790755.2790795>
- [2] Q. He, Z. Li, and X. Zhang, "Study on cloud storage system based on distributed storage systems," in *2010 International Conference on Computational and Information Sciences*, 2010, pp. 1332–1335.
- [3] B. Cha, S. Park, and J. Kim, "Design and verification of software-defined raid for hybrid cloud storage," in *2016 International Conference on Cloud Computing Research and Innovations (ICCCRI)*, 2016, pp. 128–133.
- [4] B. Fan, W. Tantisiriroj, L. Xiao, and G. Gibson, "Diskreduce: Raid for data-intensive scalable computing," *Proceedings of the 4th Annual Petascale Data Storage Workshop, PDSW '09, Held in conjunction with Supercomputing '09*, 11 2009.
- [5] Y. Yamato, "Cloud storage application area of hdd-ssd hybrid storage, distributed storage, and hdd storage," *IEEE Transactions on Electrical and Electronic Engineering*, vol. 11, no. 5, pp. 674–675, 2016. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/tee.22287>
- [6] D. Boukhelef, K. Boukhalfa, J. Boukhobza, H. Ouarnoughi, and L. Lemarchand, "Cops: Cost based object placement strategies on hybrid storage system for dbaas cloud," in *2017 17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, 2017, pp. 659–664.
- [7] D. A. Lelewer and D. S. Hirschberg, "Data compression," *ACM Computing Surveys (CSUR)*, vol. 19, no. 3, pp. 261–296, 1987.
- [8] D. T. Meyer and W. J. Bolosky, "A study of practical deduplication," *ACM Transactions on Storage (ToS)*, vol. 7, no. 4, pp. 1–20, 2012.
- [9] W. Xia, D. Feng, H. Jiang, Y. Zhang, V. Chang, and X. Zou, "Accelerating content-defined-chunking based data deduplication by exploiting parallelism," *Future Generation Computer Systems*, vol. 98, pp. 406–418, 2019.
- [10] F. Ni, X. Lin, and S. Jiang, "Ss-cdc: A two-stage parallel content-defined chunking for deduplicating backup storage," in *Proceedings of the 12th ACM International Conference on Systems and Storage*, 2019, pp. 86–96.
- [11] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "A modern primer on processing in memory," in *Emerging computing: from devices to systems: looking beyond Moore and Von Neumann*. Springer, 2022, pp. 171–243.
- [12] F. Devaux, "The true processing in memory accelerator," in *2019 IEEE Hot Chips 31 Symposium (HCS)*. IEEE Computer Society, 2019, pp. 1–24.
- [13] J. Nider, C. Mustard, A. Zoltan, J. Ramsden, L. Liu, J. Grossbard, M. Dashti, R. Jodin, A. Ghiti, J. Chauzi *et al.*, "A case study of {Processing-in-Memory} in {off-the-Shelf} systems," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, 2021, pp. 117–130.
- [14] B. Hyun, T. Kim, D. Lee, and M. Rhu, "Pathfinding future pim architectures by demystifying a commercial pim technology," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 263–279.
- [15] L.-C. Chen, S.-Q. Yu, C.-C. Ho, Y.-H. Chang, D.-W. Chang, W.-C. Wang, and Y.-M. Chang, "Rna-seq quantification on processing in memory architecture: Observation and characterization," in *2022 IEEE 11th Non-Volatile Memory Systems and Applications Symposium (NVMSA)*. IEEE, 2022, pp. 26–32.
- [16] A. Baumstark, M. A. Jibril, and K.-U. Sattler, "Processing-in-memory for databases: Query processing and data transfer," in *Proceedings of the 19th International Workshop on Data Management on New Hardware*, 2023, pp. 107–111.
- [17] C. Giannoula, I. Fernandez, J. Gómez-Luna, N. Koziris, G. Goumas, and O. Mutlu, "Towards efficient sparse matrix vector multiplication on real processing-in-memory architectures," *ACM SIGMETRICS Performance Evaluation Review*, vol. 50, no. 1, pp. 33–34, 2022.
- [18] S. Cai, B. Tian, H. Zhang, and M. Gao, "Pimpam: Efficient graph pattern matching on real processing-in-memory hardware," *Proceedings of the ACM on Management of Data*, vol. 2, no. 3, pp. 1–25, 2024.
- [19] D. Lavenier, R. Cimadomo, and R. Jodin, "Variant calling parallelization on processor-in-memory architecture," in *2020 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, 2020, pp. 204–207.
- [20] J. Gómez-Luna, I. El Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking memory-centric computing systems: Analysis of real processing-in-memory hardware," in *2021 12th International Green and Sustainable Computing Conference (IGSC)*. IEEE, 2021, pp. 1–7.
- [21] L.-C. Chen, C.-C. Ho, and Y.-H. Chang, "Uppipe: A novel pipeline management on in-memory processors for rna-seq quantification," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [22] A. S. M. Saeed and L. E. George, "Fingerprint-based data deduplication using a mathematical bounded linear hash function," *Symmetry*, vol. 13, no. 11, p. 1978, 2021.
- [23] N. Mandagere, P. Zhou, M. A. Smith, and S. Uttamchandani, "Demystifying data deduplication," in *Proceedings of the ACM/IFIP/USENIX Middleware'08 Conference Companion*, 2008, pp. 12–17.
- [24] J. Malhotra and J. Bakal, "A survey and comparative study of data deduplication techniques," in *2015 International Conference on Pervasive Computing (ICPC)*. IEEE, 2015, pp. 1–5.
- [25] S. Quinlan and S. Dorward, "Venti: A new approach to archival data storage," in *Conference on file and storage technologies (FAST 02)*, 2002.
- [26] W. Xia, Y. Zhou, H. Jiang, D. Feng, Y. Hua, Y. Hu, Q. Liu, and Y. Zhang, "{FastCDC}: A fast and efficient {Content-Defined} chunking approach for data deduplication," in *2016 USENIX Annual Technical Conference (USENIX ATC 16)*, 2016, pp. 101–114.
- [27] Y. Zhang, H. Jiang, D. Feng, W. Xia, M. Fu, F. Huang, and Y. Zhou, "Ae: An asymmetric extremum content defined chunking algorithm for fast and bandwidth-efficient data deduplication," in *2015 IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 2015, pp. 1337–1345.
- [28] A. Muthitacharoen, B. Chen, and D. Mazieres, "A low-bandwidth network file system," in *Proceedings of the eighteenth ACM symposium on Operating systems principles*, 2001, pp. 174–187.
- [29] N. Bjørner, A. Blass, and Y. Gurevich, "Content-dependent chunking for differential compression, the local maximum approach," *Journal of Computer and System Sciences*, vol. 76, no. 3-4, pp. 154–203, 2010.
- [30] VMs Archives, "Vmware virtual machines," <http://www.thoughtpolice.co.uk/vmware/>.